

HLBS 인수인계

Hierarchical Linear Blend Skinning

학습 가능한 bind pose + skin weight 기반 얼굴 표정 모델

0. Big Picture

TL;DR —

HLBS = Hierarchical LBS + 학습 가능한 (bind pose, skin weight). 얼굴 메쉬가 표정 짓는 과정을 "joint들의 transform이 skin weight로 vertex에 propagate되는" LBS로 모델링하되, identity별로 다른 bind pose와 per-id skin weight를 신경망으로 예측.

목적

- 다양한 face mesh (ICT, MF, BIWI, COMA) **topology가 전부 다른** 데이터셋을 동일한 face joint set으로 표정 학습/전이
- Identity와 expression을 disentangle해서 cross-id retargeting까지

핵심 구성요소

- **Hierarchy**: joint 트리 → FK chain으로 transform 누적
- **Per-id bind pose**: identity별로 다른 joint 위치를 신경망 예측
- **Per-vertex skin weight**: GMM prior + 신경망 hybrid

1. LBS (Linear Blend Skinning) 기초

TL;DR —

각 vertex의 deformed 위치 = "그 vertex에 영향을 주는 joint들의 transform 행렬의 가중 평균". vertex 한 점씩 적용된 결과.

개념

- Skeleton (J joints, 트리) 움직이면 mesh (N vertices) 따라 움직임
- vertex별 skin weight $W[v, j] \in [0, 1], \sum_j W[v, j] = 1$
- joint별 transform $G_j \in \mathbb{R}^{4 \times 4}$ (bind pose 대비)

LBS 식

$$\mathbf{v}'_h = \sum_{j=1}^J W[v, j] \cdot G_j \cdot \mathbf{v}_{bind, h}$$

(v_h : homogeneous coordinate $[x, y, z, 1]^\top$)

디테일

- $G_j = (\text{현재 world transform}) \cdot (\text{bind world transform})^{-1}$
- bind 위치 $\rightarrow$ joint local space $\rightarrow$ 현재 world
- LBS는 **linear** $\rightarrow$ 빠르지만 elbow/knee에서 candy-wrapper artifact (DQS가 보완)
- 코드: `models/hierarchical_lbs.py:HierarchicalLBS_FullPred.forward` 끝부분

2. Bind Pose

TL;DR —

Bind pose = "skeleton과 mesh가 처음 정렬되어 있는 reference 자세". LBS의 모든 변환은 이 자세 기준.

개념

- Mesh는 bind pose 상태로 입력 (neutral expression, T-pose 같은 것)
- 각 joint도 bind pose에서 위치/회전이 정해짐: `bind_pos[j]`, `bind_rot[j]`
- 학습은 "현재 표정 → bind 상태로 inverse → joint transform 적용" 두 단계
- LBS 핵심: **bind transform의 inverse** B_j^{-1}

디테일

- B_j : bind pose에서 joint j 의 4×4 world transform
- 코드: `model._build_B_inv(joint_pos)` — joint position만으로 B_j 만들고 (rotation은 axis 정렬), inverse 계산
- **Per-id bind pose**: ICT id마다 얼굴 크기/구조가 달라 joint 위치도 달라야 함 → `bind_pose_net` 으로 예측

3. FK Chain (Forward Kinematics)

TL;DR —

"각 joint의 world transform = parent의 world transform × 자신의 local transform"을 root에서 leaf까지 누적.

개념

- Joint 트리 구조 (root → spine → neck → head → face joints)
- Root에서 child로 propagation:

$$T_{world}[j] = T_{world}[\text{parent}(j)] \cdot T_{local}[j]$$

- 식 자체는 단순하지만 **"parent 먼저 계산" 순서 제약** 있음

디테일

- $T_{local}[j] = T_{bind_local}[j] \cdot T_{delta}[j]$
- T_{bind_local} : bind pose에서 부모 기준 상대 transform (한 번 precompute, buffer 저장)
- T_{delta} : 학습/예측되는 motion (rotation 6D + optional translation)
- **Rotation parameterization**: 6D continuous [Zhou et al. CVPR 2019] → Euler gimbal lock / quaternion antipodal 회피
- 코드: `_chain_hierarchy(T_delta)`
- `process_order` buffer: helper reparenting 후 `parent_idx > self_idx` 발생 → 순차 iteration 불가 → topological sort 필요 (`utils/rig_loader.py` fallback)

4. Skin Weights W

TL;DR —

$W[v, j]$ = vertex v 가 joint j 에 얼마나 묶여있는지 ($\in [0, 1]$, $\sum_j W = 1$). 신경망 출력 + GMM Gaussian prior의 **hybrid**.

개념

- 매 forward마다 model이 vertex별 W 예측
- Sparse property (vertex는 보통 4-8 joint에만 영향) → softmax + Gauss prior로 강제

Hybrid 식

$$W[v, j] = \text{softmax}_j (\text{logit}_{\text{net}}[v, j] + \text{logit}_{\text{gauss}}[v, j])$$

- `logit_net`: 신경망 (DiffusionNet / Linear encoder)
- `logit_gauss`: GMM prior, vertex가 joint center에 가까울수록 logit 큼

디테일

- GMM prior 종류: **additive** ($-\|v - p_j\|^2 / (2\sigma_j^2)$) vs. **residual** (우리 main 사용, bounded 영역 안에서만 net이 prior 보정)
- σ 는 per-joint 학습 (`log_sigma`), `sigma_targets.npy` regularization target
- **face_joint_idx mask**: face 영역 vertex만 face joints softmax. non-face는 `base_joint` (보통 `skull_root`) 단일 weight=1 routing → mesh 외곽(목, 뒤통수)이 face motion에 안 끌려감
- 코드: `_get_skinning_weights()`, `sigma_shrink_loss()`

5. 학습 가능한 것 / 고정된 것

TL;DR —

고정: joint 트리, $J=66$, parent_idx, helper reparenting, home_vidx. **학습:** per-id bind pose, per-vertex skin weight, expression latent z_{exp} , joint transform delta T .

Component	Fixed / Learned	Source
Joint hierarchy (parent_idx)	Fixed	rig_info_*.json
Bind pose target (mean)	Fixed	Maya export → rig.bind_pos_dict
Per-id bind pose	Learned (predicted)	bind_pose_net
Skin weights	Learned	skin_weight_net + GMM prior
Expression encoder	Learned	lbs_exp_z_model
Joint transform per frame	Learned	lbs_pose_model
σ (Gaussian width)	Learned (per-joint scalar)	log_sigma

6. Helper Joints (Option A)

TL;DR —

Skull_Root 근처에 frozen이었던 **19개 joint**를 brow/eye/cheek/mouth로 reparent해서 face motion expressiveness ↑. helper joint 위치는 **parent + small residual**로 reparameterize.

개념

- 원래 ICT/MF rig에는 face motion과 무관한 joint 다수 (ears, cranium 등)
- 그 중 19개를 face 영역 (eye_socket, cheek_bone, maxilla 등)으로 parent 변경
- helper joint position 직접 학습하면 발산 → **residual parameterization**:

$$\mathbf{p}_{helper} = \mathbf{p}_{parent} + \Delta_{helper}$$

- `bind_pose_net` 의 helper output을 residual로 해석
- $L_{helper_residual} = \|\Delta\|^2$ L2 reg → helper가 parent 근처에 머물

디테일

- 좌우 helper pair (8쌍): $L_{mirror} = \text{MSE}(\text{pos}[L], \text{mirror}_x(\text{pos}[R]))$ — bilateral 대칭 강제
- L_{bind_reg} 에서 helper **제외** — per-id GT가 parent 위치 그대로라 MSE면 자유도 0
- 코드: `models/hierarchical_lbs.py:_get_bind_pose` 끝부분, `train_hlbs.py:L_helper_residual / L_mirror`

7. Per-id Bind Pose Supervision (Phase B v3)

TL;DR —

각 identity의 anatomical landmark를 미리 mesh vertex index로 매핑해두고, per-id mesh의 그 vertex 위치 + 고정 offset으로 per-id bind pose 생성. 학습 시 $L_{\text{bind_reg}}$ target.

개념

- 같은 표정이라도 ICT id마다 얼굴 크기/구조 → joint 위치 달라야 함
- 사용자 manual SPECS 정의: "Top_Lip joint의 home vertex = DTU3D landmark 50 nearest mesh vertex"

각 id에 대해:

$$\text{bind_pos}_{id}[j] = V_{id}[\text{home_vidx}[j]] + (\text{bind_pos}_{mean}[j] - V_{mean}[\text{home_vidx}[j]])$$

디테일

- Mean mesh에서 `(mean bind_pos - mean home_vertex)` = constant offset (joint별)
- 각 id에선 그 id의 home vertex 위치 + 위 offset → per-id bind pose
- Helper joint는 parent의 per-id bind pose 그대로 상속
- 코드: `tools/precompute/compute_joint_home_vidx_{ict,mf}.py` + `precompute_per_id_bind_pos_v2.py`
- Cache: `nfs_features_seg/{id_name}_bind_pos_landmark.npy`

8. Loss Functions 요약

TL;DR —

Recon (vertex/normal MSE) + Bind pose supervision (per-id) + Skin weight regularization (GMM/locality)의 다층 구조.

Loss	식	역할
recon-lbs	$\text{MSE}(\hat{v}, v_{gt})$	기본 reconstruction
recon-normal	$1 - \cos(\hat{n}, n_{gt})$	표면 smoothness
recon-neu	$\text{MSE}(\hat{v}_{neu}, v_{tpl})$	identity-only 출력 보존
$L_{\text{bind_reg}}$	$\text{MSE}(\mathbf{p}_j, \mathbf{p}_j^{(id)})$	per-id bind pose 학습
$L_{\text{helper_residual}}$	$ \Delta_{\text{helper}} ^2$	helper 발산 방지
L_{mirror}	$\text{MSE}(\mathbf{p}_L, \text{mirror}_x(\mathbf{p}_R))$	L/R helper 대칭
L_{dist}	$\sum W[v, j] \cdot v - \mathbf{p}_j ^2$	skin weight locality
$L_{\text{net_center}}$	$\text{MSE}(\sum W \cdot v, \mathbf{p}_j)$ or $W \cdot d_{geo}^2$	weight를 joint 주변 집중
L_{σ}	$\max(0, \log \sigma - \log \sigma_{tgt})^2$	σ 폭주 방지
$L_{\text{cross_retarget}}$	$\text{MSE}(\text{retarget}(A \rightarrow B), \text{GT}_B)$	id-exp disentanglement
$L_{\text{cross_cyclic}}$	$\text{MSE}(\text{retarget}(B \rightarrow A \text{ of } \hat{v}_{tgt}), v_{src})$	cycle consistency

9. Subsample Augmentation (mix4)

TL;DR —

매 batch마다 4가지 sampling mode 중 1/4 확률로 random pick → resolution-invariance + 영역별 학습 강도 다양화.

4가지 모드 (각 1/4)

1. **Full mesh** — 모든 N vertex 사용
2. **FPS** — PyTorch3D `sample_farthest_points`, 균등 spatial coverage
3. **Random** — uniform random sampling
4. **Importance-strict** — brow/eye/nose/mouth landmark 영역에서만 (zero outside)

Per-batch dynamic ratio

- Random / FPS: ratio $\sim \mathcal{U}[0.1, 0.5]$
- Importance: contour vertex 수로 cap

효과

- 단일 resolution overfit 방지
- 표정에 중요한 영역 (입, 눈) supervision 강도 ↑
- Inference 시 다양한 mesh density에 robust

10. Cross-retarget (Track B)

TL;DR —

"id A의 표정을 id B의 mesh에 적용한 결과가 id B의 그 표정 GT와 일치"라는 supervision으로 **identity-expression disentanglement** 강제.

동작

- 같은 expression z_{exp} 샘플, 두 id에 적용
- **GT**: $(\text{ICT}(A, 0), \text{ICT}(A, z_{exp}), \text{ICT}(B, 0), \text{ICT}(B, z_{exp}))$
- **Forward retarget**: `pred_tgt = model.retarget(src_neu(A), src_def(A), tgt_neu(B))`
 - expression은 source에서, identity (W , bind_pose)는 target에서
- $L_{\text{cross_retarget}} = \text{MSE}(\text{pred_tgt}, \text{ICT}(B, z_{exp}))$

Cyclic 확장 (Run 4)

- **Reverse retarget**: `pred_src_recon = retarget(tgt_neu(B), pred_tgt, src_neu(A))`
- $L_{\text{cross_cyclic}} = \text{MSE}(\text{pred_src_recon}, \text{src_def}(A))$
- → **bijective encoding** 강제

11. 학습 파이프라인 한 번에

TL;DR —

Input mesh $\rightarrow$ bind_pose_net (per-id joint 위치) + expression encoder + pose model $\rightarrow$ FK chain $\rightarrow$ skin weight (net + GMM)
 $\rightarrow$ LBS $\rightarrow$ pred vertex.

```
[input]
src_template(neutral) + src_vertices(expression) + (NFS feature 256d)
↓
[bind_pose_net]  $\rightarrow$  joint_pos [J, 3] (per-id)
↓
[B_inv (bind pose inverse)]
[expression encoder]  $\rightarrow$  z_exp [d]
↓
[pose model]  $\rightarrow$  6D rot + trans per joint
↓
[FK chain via process_order]  $\rightarrow$  T_world [J, 4, 4]
↓
G_j = T_world · B_inv ← FK + skin
↓
[skin_weight_net]  $\rightarrow$  logit_net + [GMM prior]  $\rightarrow$  logit_gauss
└─ softmax_j(sum) ─┘
    ↓
    W [B, V, J]
    ↓
[LBS]:  $v'_h = \sum_j W[v, j] \cdot G_j \cdot v_{bind\_h} \rightarrow pred\_v [B, V, 3]$ 
```

12. 코드 위치 참고

TL;DR —

Rig 로딩 → 모델 → 학습 루프 → precompute → vis 순으로 정리. Phase B v3 (per-id bind pose) 관련 파일은 `tools/precompute/` 아래.

모듈	파일
Rig 로딩, <code>process_order</code>	<code>utils/rig_loader.py</code>
모델 (LBS, bind pose, skin weight)	<code>models/hierarchical_lbs.py</code>
학습 루프 (loss, sampling, vis)	<code>train_hlbs.py</code>
Per-id bind pose precompute (Phase B v3)	<code>tools/precompute/precompute_per_id_bind_pos_v2.py</code>
Helper joint setup	<code>tools/precompute/setup_helper_joints.py</code>
Joint home vertex spec	<code>tools/precompute/compute_joint_home_vidx_{ict,mf}.py</code>
Sampling vis	<code>tools/vis/vis_subsample.py</code>
Bind pose Maya export	<code>tools/precompute/export_pred_joints_maya.py</code>



Questions?